

アッカーマン関数 逆アッカーマン関数

アッカーマン関数，逆アッカーマン関数

1. 概要

アッカーマン関数はある漸化式によって定義される 2 変数関数で，極端に速く増加するという特徴があります．競技プログラミングでは，その逆関数のようなものである**逆アッカーマン関数**が，Union-Find や静的なモノイド列の区間積クエリの計算量評価に現れることが有名です．

(逆) アッカーマン関数については，定義の細部を変更した変種もよく用いられ，それらも (逆) アッカーマン関数と呼ばれることがあります．本講座ではこれらの複数の定義についても確認し，それらの関係や，定義の異なる逆アッカーマン関数同士に高々定数の違いしかないことなどを確認します．

本講座は AtCoder Algorithm Lectures 内の他の講座で逆アッカーマン関数を扱うための準備を目的としています．本講座ではこれらの関数を，アルゴリズムやその計算量解析に用いることはありません．

2. 前提知識

特にありません．

3. アッカーマン関数

3.1. アッカーマン関数の定義

【定義 1】

非負整数の組 (k, n) に対して定義され正整数値をとる関数 $A_k(n)$ を，次の式によって定義する (k, n は非負整数)．

$$\begin{aligned} A_0(n) &= n + 1, \\ A_{k+1}(0) &= A_k(1), \\ A_{k+1}(n+1) &= A_k(A_{k+1}(n)). \end{aligned}$$

この関数を**アッカーマン関数** (Ackermann function) という． $A_k(n)$ を $A(k, n)$ とも書く．

同じ関数の n 回合成 $\underbrace{f \circ f \circ \cdots \circ f}_{n \text{ 個}}(x)$ を $f^{(n)}(x)$ と書くことにすれば，定義式は，次のようにも表せます．

$$\begin{aligned} A_0(n) &= n + 1, \\ A_{k+1}(n) &= A_k^{(n+1)}(1). \end{aligned}$$

小さな k, n に対する $A_k(n)$ の値は以下の表のとおりです．

$k \backslash n$	0	1	2	3	4
0	1	2	3	4	5
1	2	3	4	5	6
2	3	5	7	9	11
3	5	13	29	61	125
4	13	65533	$2^{65536} - 3$	$2^{2^{65536}} - 3$	$2^{2^{2^{65536}}} - 3$

$k \geq 1$ のとき第 k 行では，ひとつ右の列に進むたびに A_{k-1} への代入を行った値になっています．

$$A_0(n) = n + 1, \quad A_1(n) = n + 2, \quad A_2(n) = 2n + 3, \quad A_3(n) = 2^{n+3} - 3$$

が成り立ちます． $k = 4$ については，

$$A_4(n) = 2^{2^{\cdots^2}} - 3 \quad (\text{右辺は } n + 3 \text{ 個の } 2 \text{ からなる})$$

が成り立ちます。なお、このように同じ数の指数をとる操作を反復したものは**テトレーション** (tetration) と呼ばれます。

- $A_4(2)$ は 19729 桁.
- $A_4(3)$ は桁数が 10^{19728} 程度 (「桁数の桁数」は 5 桁).
- $A_4(4)$ は「桁数の桁数」が 10^{19728} 程度.

となっており、 k, n が小さい値であるにも関わらず $A_k(n)$ は非常に大きな値になることが分かります。

3.2. アッカーマン関数の変種 (1)

アッカーマン関数の定義については定義の細部を変更した変種もよく用いられ、それらもアッカーマン関数と呼ばれることがあります。

例えば、逆アッカーマン関数を用いた Union-Find の計算量上界を証明した[文献 3](#) では、次のような定義が用いられています。3.1 で定義したアッカーマン関数と区別するため、この関数をここでは $A_k(n)$ ではなく $B_k(n)$ と書くことにします。

【定義 2】

非負整数 k と正整数 n に対して正整数値をとる関数 $B_k(n)$ を、次の式によって定義する (k は非負整数で n は正整数)。

$$\begin{aligned} B_0(n) &= 2n, \\ B_k(1) &= 2, \\ B_{k+1}(n+1) &= B_k(B_{k+1}(n)). \end{aligned}$$

小さな k, n に対する $B_k(n)$ の値は以下の表のとおりです。

$k \backslash n$	1	2	3	4	5	6
0	2	4	6	8	10	12
1	2	4	8	16	32	64
2	2	4	16	65536	2^{65536}	$2^{2^{65536}}$

次の命題が示すように、これは 3.1 で扱ったアッカーマン関数に簡単な平行移動を行っただけのものです。 $A_3(n) = 2^{n+3} - 3$ のような「+3, -3」といった端数がなくなるように調整したものと考えてよいでしょう。

【命題 3】

$k \geq 0, n \geq 3$ に対して

$$B_k(n) = A_{k+2}(n-3) + 3$$

が成り立つ。

細部を省略しつつ概要のみ述べます。まず

- $k = 0$ の場合に成り立つこと。
- $B_k(2) = 4$ が成り立つこと。

を示しておきます。

主張を k に関する数学的帰納法により証明します。 k の場合の主張を認めて、 $k+1$ の場合を示します。この際さらに、 n について数学的帰納法を使います。まず $n = 3$ のときは、

$$B_{k+1}(3) = B_k(B_{k+1}(2)) = B_k(4) = A_{k+2}(1) + 3 = A_{k+3}(0) + 3$$

となり示されます。 $n \geq 3$ について、 $(k+1, n)$ での成立を仮定すると、

$$\begin{aligned} B_{k+1}(n+1) &= B_k(B_{k+1}(n)) = A_{k+2}(B_{k+1}(n)-3) + 3 \\ &= A_{k+2}(A_{k+3}(n-3)) + 3 = A_{k+3}(n-2) + 3 \end{aligned}$$

となり $(k+1, n+1)$ の場合が示されます。以上で **【命題 3】** が示されました。 ■

3.3. アッカーマン関数の変種 (2)

上に挙げた以外にもアッカーマン関数や逆アッカーマン関数の変種はいくつかあります。そのうち特に Union-Find の解説で引用されていることの多い、[文献 4](#) の場合に触れておきます。

[文献 4](#) では次の漸化式で定まる $C_k(n)$ を ($A_k(n)$ という記号で) アッカーマン関数の変種として導入しています。

【定義 4】

非負整数 k, n に対して $C_k(n)$ を次の式によって定義する.

$$\begin{aligned} C_0(n) &= n + 1, \\ C_{k+1}(n) &= C_k^{(n+1)}(n). \end{aligned}$$

かなり $A_k(n)$ に似た定義式ですが, $C_k^{(n+1)}$ に代入される値が 1 ではなく n になっていることに注意してください.

$C_k(n)$ は $A_k(n)$ の単純な変数変換にはならないのですが, この場合にも次のように, $C_k(n)$ と $A_k(n)$ は似た増加速度を持っていることが分かります.

【命題 5】

k を非負整数, n を正整数とすると, 次が成り立つ.

$$A_k(n) \leq C_k(n) \leq A_{k+1}^{(2)}(n) \leq A_{k+2}(n).$$

数学的帰納法で示します. $A_k(n) \leq C_k(n)$ についてはかなり易しいので省略します.

$C_k(n) \leq A_{k+1}^{(2)}(n)$ を示します. $k = 0, 1$ の場合は, $C_0(n) = n + 1$, $C_1(n) = 2n + 1$ などから直接確かめられます.

k を正整数として, 任意の正整数 n について $C_k(n) \leq A_{k+1}^{(2)}(n)$ が成り立つことを仮定します. $C_{k+1}(n) \leq A_{k+2}^{(2)}(n)$ を示します.

$$C_{k+1}(n) = C_k^{(n+1)}(n) \leq A_{k+1}^{(2(n+1))}(n) \leq A_{k+1}^{(3n+1)}(1) = A_{k+2}(3n)$$

です. 1 つめの $\leq$ は帰納法の仮定から分かり, 2 つめの $\leq$ では $A_{k+1}(n) \leq A_{k+1}^{(2)}(n-1) \leq \dots \leq A_{k+1}^{(n)}(1)$ のようにして分かります.

さらに $3n \leq A_3(n) \leq A_{k+2}(n)$ であることから,

$$C_{k+1}(n) \leq A_{k+2}(3n) \leq A_{k+2}(A_{k+2}(n)) = A_{k+2}^{(2)}(n)$$

となります. 帰納法により任意の k, n について $C_k(n) \leq A_{k+1}^{(2)}(n)$ が成り立つことが示されました.

最後に $A_{k+1}^{(2)}(n) \leq A_{k+2}(n)$ は,

$$A_{k+1}^{(2)}(n) \leq A_{k+1}^{(3)}(n-1) \leq \cdots \leq A_{k+1}^{(n+1)}(1) = A_{k+2}(n)$$

のように示すことができます. ■

4. 逆アッカーマン関数

逆アッカーマン関数の定義について説明します. 逆アッカーマン関数についても, (アッカーマン関数の定義の違いを除いても) よく使われる定義がいくつか存在します.

4.1. 逆アッカーマン関数の定義

【定義 6】

正整数 x に対して, **逆アッカーマン関数** (inverse Ackermann function) $\alpha(x)$ を次のように定義する.

$$\alpha(x) = \min\{k \geq 0 \mid A_k(k) \geq x\}.$$

逆アッカーマン関数は, **アッカーマン逆関数**とも呼ばれています. 一方で, 関数論や集合論などで標準的に用いられる「逆関数」とは少し意味が異なることには注意してください.

上で求めたアッカーマン関数の値から, $\alpha(x)$ については次のような具体値が分かります.

- $x = 1$ ならば $\alpha(x) = 0$.
- $1 < x \leq 3$ ならば $\alpha(x) = 1$.
- $3 < x \leq 7$ ならば $\alpha(x) = 2$.
- $7 < x \leq 61$ ならば $\alpha(x) = 3$.
- $61 < x \leq 2^{2^{65536}} - 3$ ならば $\alpha(x) = 4$.

アッカーマン関数は k について極端に速く増大することから, 逆アッカーマン関数は極端にゆっくりと増大する関数です. 競技プログラミングで実際に入力として現れる範囲の正整数 n については, 通常 $\alpha(n) \leq 4$ とみなして差し支えありません.

4.2. $A_k(0)$ などによる定義

逆アッカーマン関数については、他にも定義方法がありますが、通常それらの差は $O(1)$ で、特に計算量オーダーの文脈では区別の必要がありません。

ひとつの方法は、 $A_k(k)$ の代わりに $A_k(0)$ や $A_k(1)$ などを用いるものです。ここでは $A_k(0)$ の場合に 【定義 6】 との差が $O(1)$ であることを確認します。

【命題 7】

正整数 x に対して $f(x)$ を次のように定義する。

$$f(x) = \min\{k \geq 0 \mid A_k(0) \geq x\}.$$

このとき正整数 x に対して次が成り立つ。

$$\alpha(x) \leq f(x) \leq \alpha(x) + 2.$$

$A_k(0) \leq A_k(k)$ であるから $\alpha(x) \leq f(x)$ は明らかです。 $f(x) \leq \alpha(x) + 2$ を示します。

そのためには $A_k(k) \geq x$ を仮定して $A_{k+2}(0) \geq x$ を示せばよいです。これは次の計算により示されます：

$$A_{k+2}(0) = A_{k+1}(1) = A_k(A_{k+1}(0)) = A_k(A_k(1)) \geq A_k(k) \geq x.$$

ここで最後から 2 番目の不等号で、 $A_k(1) \geq k$ を用いました。この不等式も帰納法で容易に証明できます。 ■

他に、 $A_k(1)$ や $B_k(3)$ を用いて逆アッカーマン関数を定義しても、【命題 7】 のような結果が成り立ちます。また 【命題 5】 から、 $C_k(1)$ を用いた場合（文献 4 における逆アッカーマン関数の定義）でも同様です。

4.3. アッカーマン関数を用いず直接定義する方法

文献によっては逆アッカーマン関数を、見た目上はアッカーマン関数を用いずに、直接定義しています。このような方法と、アッカーマン関数を用いる定義との関係について確認します。

【定義 8】

非負整数 k と正整数 x に対して, $g_k(x)$ を次の式によって定義する.

$$\begin{aligned}g_0(x) &= \left\lceil \frac{x}{2} \right\rceil, \\g_k(1) &= 0, \\g_{k+1}(x) &= 1 + g_k(g_k(x)) \quad (k \geq 0, x \geq 2).\end{aligned}$$

定義式は, 次のように書くこともできます.

$$g_{k+1}(x) = \min \left\{ n \geq 0 \mid g_k^{(n)}(x) = 1 \right\}.$$

小さな k に対する $g_k(x)$ は以下の通りです.

$$\begin{aligned}g_0(x) &= \left\lceil \frac{x}{2} \right\rceil, \\g_1(x) &= \lceil \log_2 x \rceil, \\g_2(x) &= \log_2^* x.\end{aligned}$$

▶ $\log_2^* x$ とは

次の命題の意味で, $g_k(-)$ は $B_k(-)$ の逆関数にあたるものです (B は 3.2 節で定義されたアッカーマン関数の変種です).

【命題 9】

非負整数 k と 2 以上の整数 x に対して次が成り立つ.

$$g_k(x) = \min \{ n \geq 1 \mid B_k(n) \geq x \}.$$

数学的帰納法により証明します. $k = 0$ の場合は容易です.

k の場合を仮定して, $k + 1$ の場合を示します. 特に, $g_k(2) = 1$ であることや $g_k(x) < x$ を仮定します. さらに, $k + 1$ の場合について x に関する帰納法により示します. $x = 2$ の場合は明らかです. $x \geq 3$ として x 未満の場合を仮定すると, $n \geq 2$ について成り立つ次の同値変形

$$\begin{aligned}
B_{k+1}(n) \geq x &\iff B_k(B_{k+1}(n-1)) \geq x \\
&\iff B_{k+1}(n-1) \geq g_k(x) \\
&\iff n-1 \geq g_{k+1}(g_k(x))
\end{aligned}$$

により, $\min\{n \mid B_{k+1}(n) \geq x\} = 1 + g_{k+1}(g_k(x)) = g_{k+1}(x)$ となります. ■

【定義 10】

正整数 x に対して, $g(x)$ を次の式によって定義する.

$$g(x) = \min\{k \mid g_k(x) \leq 3\}.$$

【定義 8】 と 【定義 10】 から, アッカーマン関数 $A_k(n)$ や $B_k(n)$ を持ち出すことなく $g(x)$ を定義できていることに注意してください.

次の命題の意味で, $g(x)$ は $B_k(3)$ の逆関数にあたるものです. したがってこの $g(x)$ も $\alpha(x)$ との差が $O(1)$ で, $g(x)$ はアッカーマン逆関数の変種と考えることができます.

【系 11】

非負整数 k と 2 以上の整数 x に対して, 次が成り立つ.

$$g(x) = \min\{k \mid B_k(3) \geq x\}.$$

【命題 9】 と 【定義 10】 から明らかです. ■

5. 関連問題

1. https://atcoder.jp/contests/kupc2012pr/tasks/kupc2012pr_1
2. <https://projecteuler.net/problem=282>

6. まとめ

本講座では, アッカーマン関数および逆アッカーマン関数の定義を確認しました. また小さな k, n に対する具体値を確認することで, アッカーマン関数は極端に速く増加すること, 逆アッカーマン関数は極端にゆっくりと増大することが何となく分かったと思います.

アッカーマン関数および逆アッカーマン関数には、文献による定義揺れも散見されます。そのため、ある文献で証明されている結果が、他の文献での定義でも成り立つのか分からないということがあり、私も理解にかなり苦労したことがあります。本講座ではそのような定義揺れについても一定の解説をしました。このことにより、本講座を読んだ方が、アッカーマン関数や逆アッカーマン関数を含む文献を学習しやすくなることを期待しています。

逆アッカーマン関数は競技プログラミングでは、Union-Find や、静的なモノイド列に対する区間積クエリの計算量評価に現れることが有名です。これらの話題については以下の講座を参照してください。

- [Union-Find の計算量上界](#)
- [静的な列の区間積クエリ](#)

7. 参考文献

1. Wikipedia. アッカーマン関数. <https://ja.wikipedia.org/wiki/アッカーマン関数>. (閲覧日: 2026-03-30).
2. Wikipedia. Ackermann function. https://en.wikipedia.org/wiki/Ackermann_function. (閲覧日: 2026-03-30).
3. R. E. Tarjan. Efficiency of a Good But Not Linear Set Union Algorithm. Journal of the ACM. Vol. 22. No. 2. pp. 215–225. 1975. DOI: 10.1145/321879.321884. <https://doi.org/10.1145/321879.321884>.
4. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. アルゴリズムイントロダクション 第3版 総合版. (翻訳: 浅野 哲夫, 岩野 和生, 梅尾 博司, 山下 雅史, 和田 幸一). 近代科学社. 2013. https://www.kindaikagaku.co.jp/book_list/detail/9784764904088/.
5. Gabriel Nivasch. Inverse Ackermann without pain. <https://www.gabrielnivasch.org/fun/inverse-ackermann>. (閲覧日: 2026-03-30).
6. Wikipedia. テトレーション. <https://ja.wikipedia.org/wiki/テトレーション>. (閲覧日: 2026-03-30).
7. Wikipedia. 反復対数. <https://ja.wikipedia.org/wiki/反復対数>. (閲覧日: 2026-03-30).

文献 3 は逆アッカーマン関数を用いた Union-Find の計算量上界を証明した論文で、本講座における $B_k(n)$ をアッカーマン関数として用いています。文献 4 はアルゴリズムの定番教科書で、Union-Find の計算量上界として、本講座における $C_k(1)$ の逆関数を逆アッカーマン関数としています。

文献 5 はアッカーマン関数を経由することなく逆アッカーマン関数について論じている文献であり、アッカーマン関数を利用せずに Union-Find や区間積クエリの計算量について解説している他、「標準的な逆アッカーマン関数」の定義について提案しています。本講座における $g_k(x)$, $g(x)$ の定義がおおよそこの文献における $\alpha_k(x)$, $\alpha(x)$ に対応します。

トップページ：[AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など：[Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

各サービス

[AtCoderとは？](#)

[AtCoder](#) 

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)